Directed Graphs

Prof. Rohit Pal Singh

Department of Computer Science

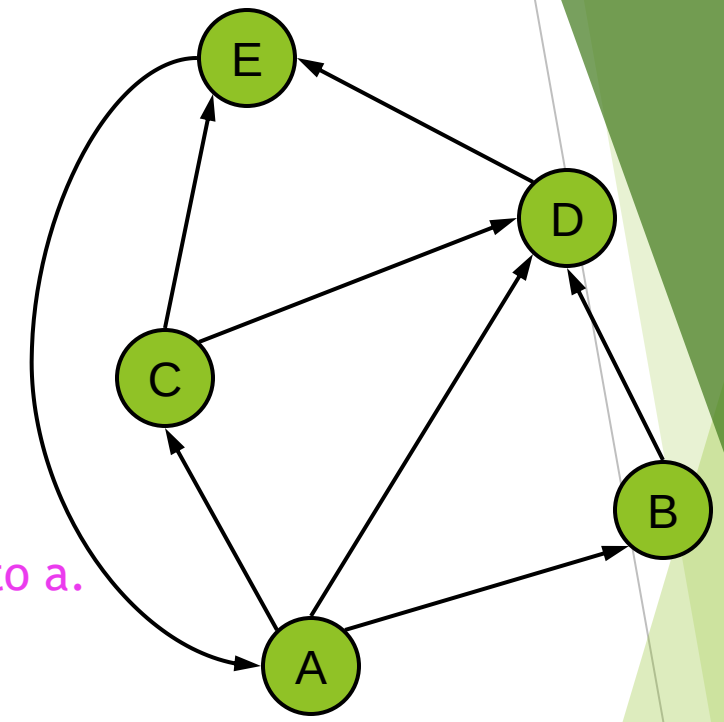
University of Wisconsin, Milwaukee

November 20, 2024

Readings: Chapter 6.4

Digraphs

- A **digraph** is a graph whose edges are all directed
  - Short for "directed graph"



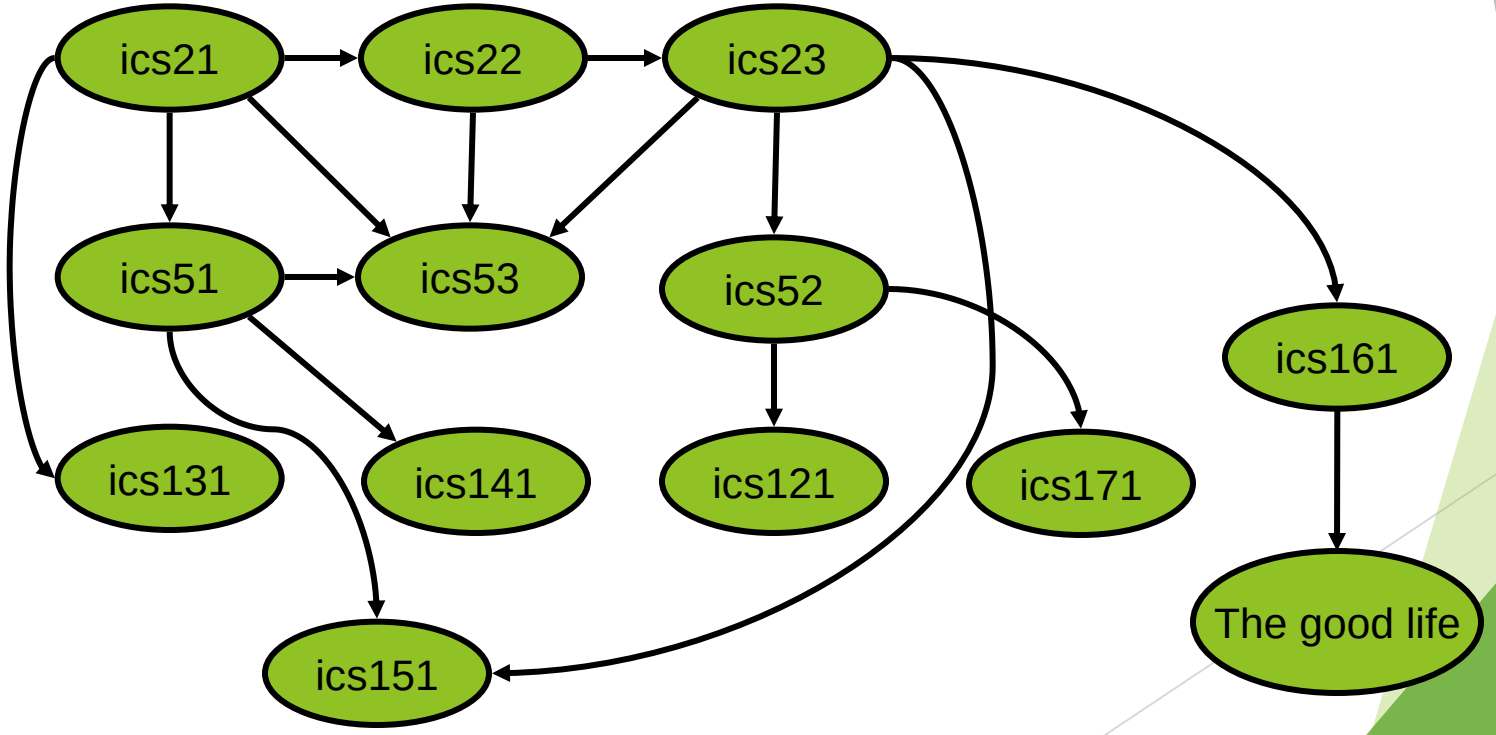
- A graph G=(V,E) such that
  - Each edge goes in one direction:

    Edge (a,b) goes from a to b, but not b to a.
- If G is simple, $m \leq n(n-1)$.
- If we keep in-edges and out-edges in separate adjacency lists, we can perform listing of the sets of in-edges and out-edges in time proportional to their size.

Digraph Applications

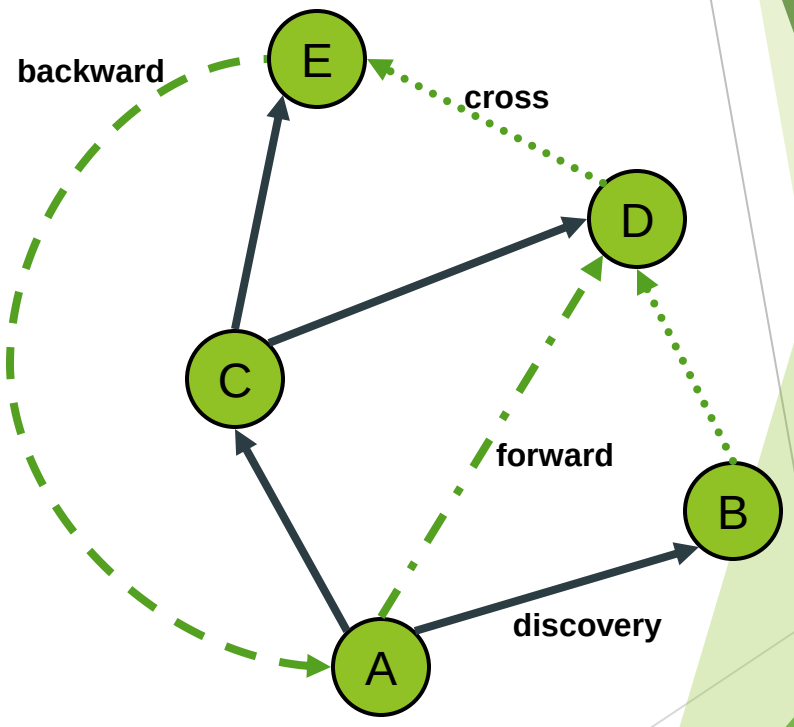
Examples: one-way streets, flights, task scheduling, course-taking

- Scheduling: edge (a, b) means task *a* must be completed before *b* can be started



Directed DFS/BFS

- We can specialize the traversal algorithms (DFS and BFS) to digraphs by traversing edges only along their direction
- In the directed DFS algorithm, we have four types of edges
  - discovery edges
  - back edges
  - forward edges
  - cross edges
- The directed BFS algorithm has three types of edges:
  - discovery edges
  - back edges
  - cross edges

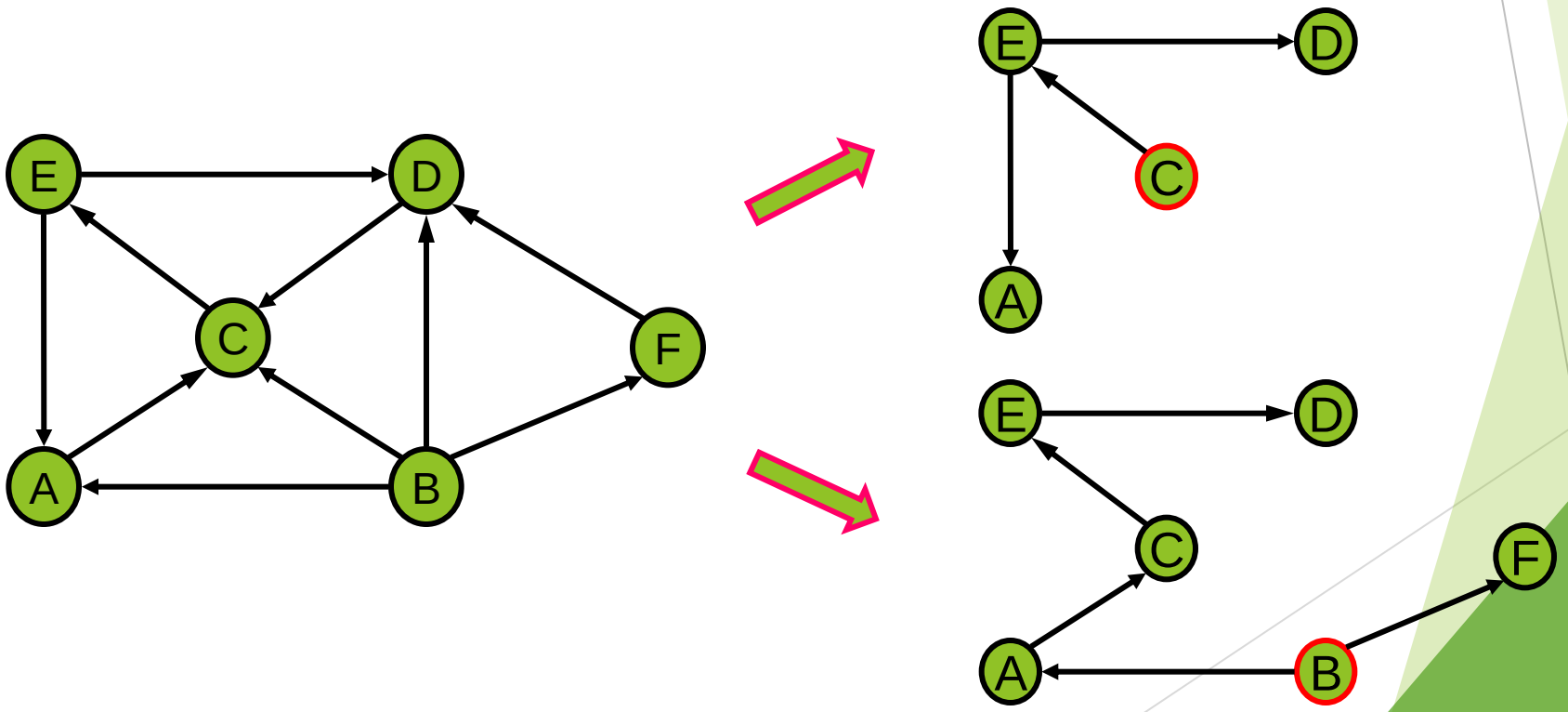


DFS

Reachability

- u can reach v in a directed G (or v is reachable from u):

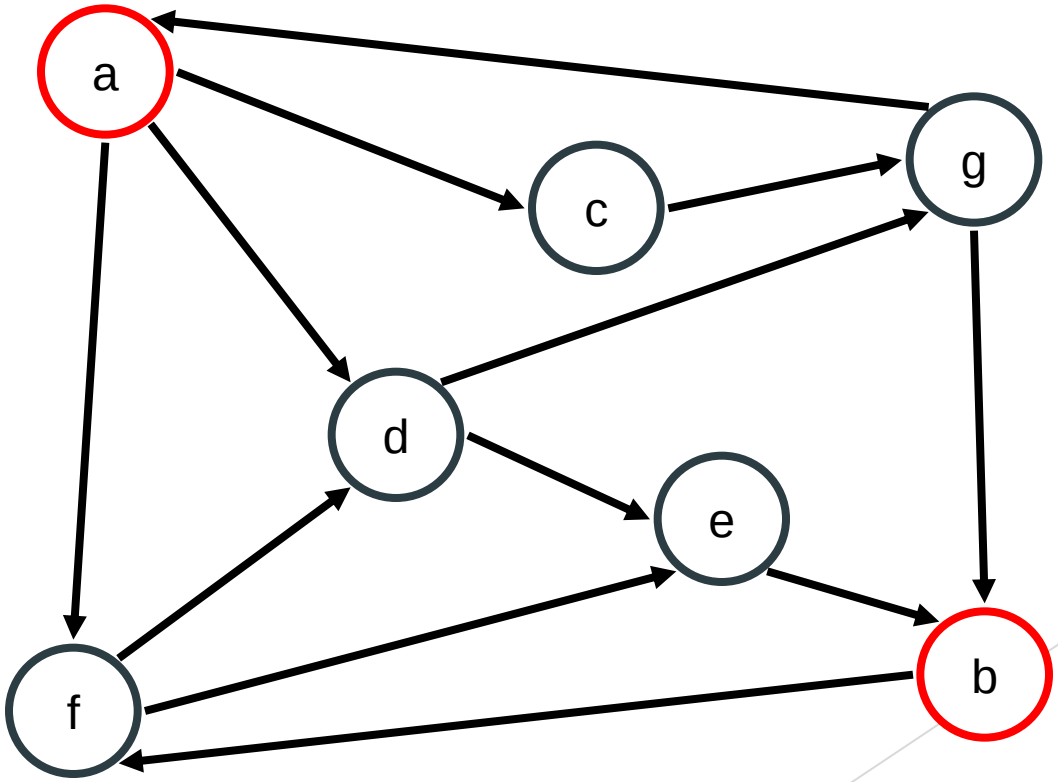
  if there is a directed path from u to v.
- DFS tree rooted at v: contains all vertices reachable from v via directed paths. Running time O(n+m).



Strong Connectivity

- Each vertex can reach all other vertices (for example, *a* & *b*)
- BFS/DFS can be used to test if G is strongly connected or not.

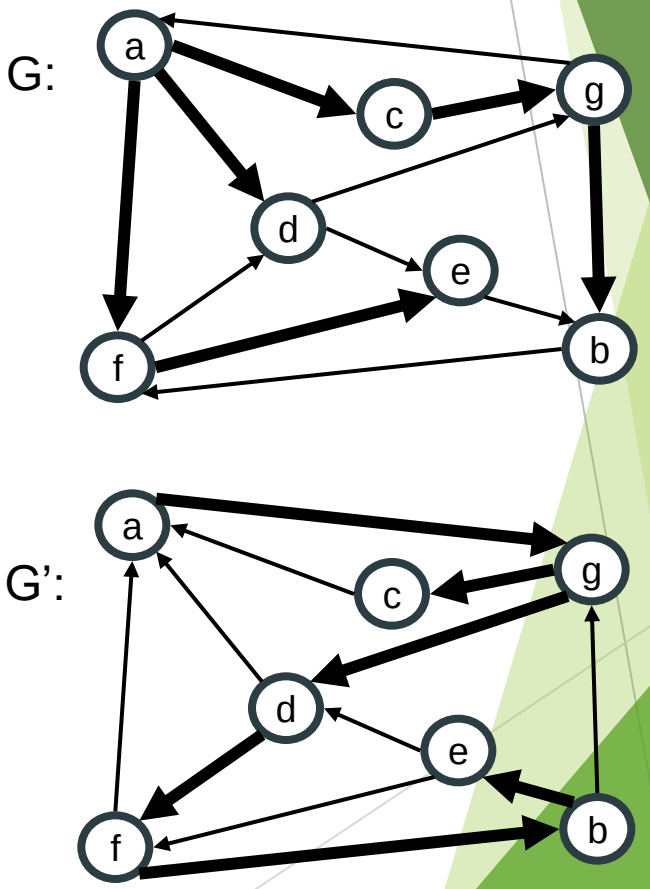
  Total time: O(n(n+m)).



More Efficient Algorithm for Strong Connectivity

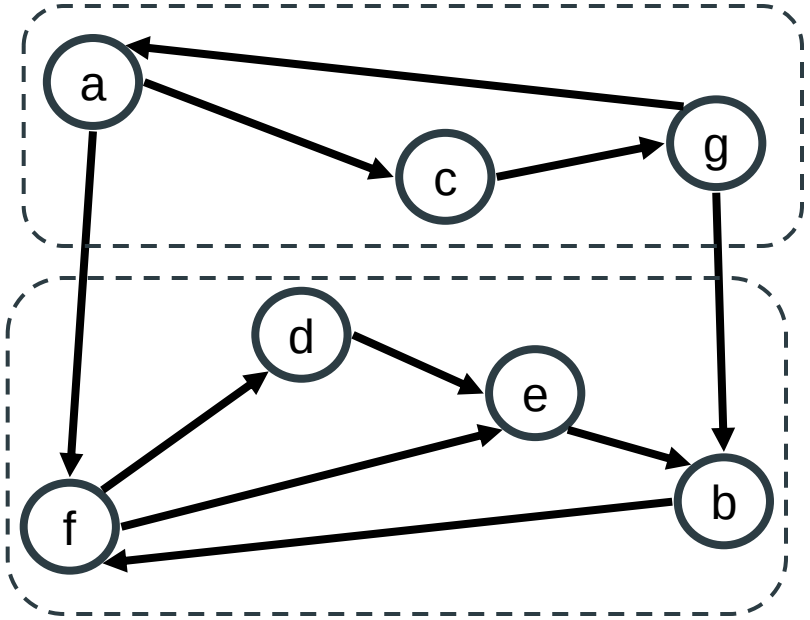
- Pick a vertex v in G.
- Perform a DFS from v in G.
  - If there's a w not visited, print "no".
- Let G' be G with edges reversed.
- Perform a DFS from v in G'.
  - If there's a w not visited, print "no".
  - Else, print "yes".

- Running time: O(n+m).



Strongly Connected Components

- Maximal subgraphs such that each vertex can reach all other vertices in the subgraph
- Can also be done in O(n+m) time using DFS, but is more complicated

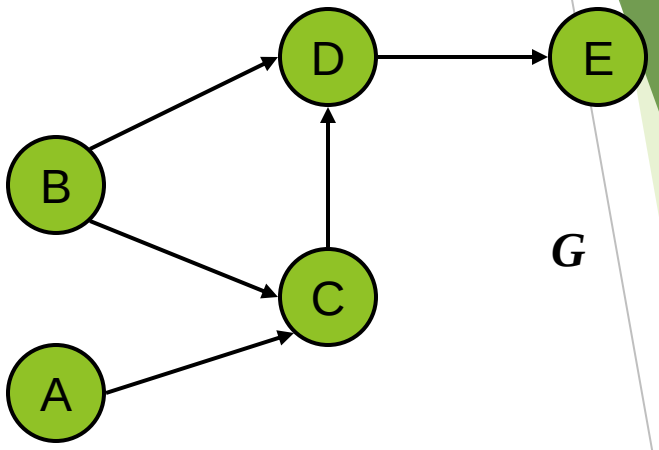


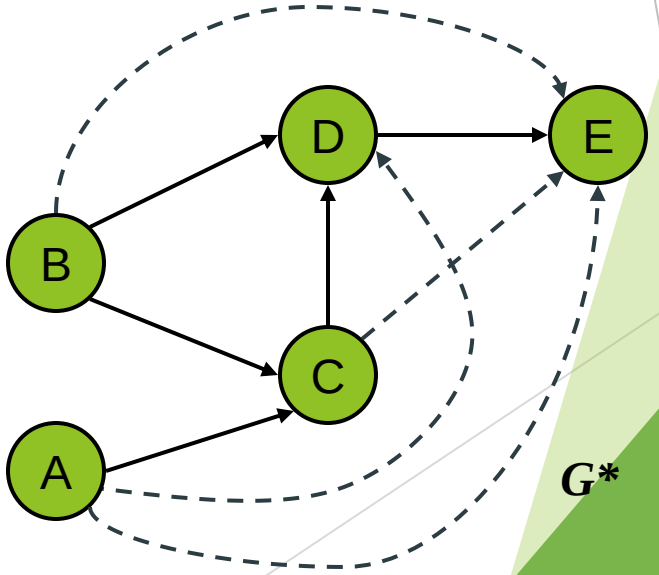
{ a , c , g }

{ f , d , e , b }

Transitive Closure

- Given a digraph G, the transitive closure of G is the digraph G^* such that
  - G^* has the same vertices as G
  - if G has a directed path from u to v ($u \neq v$), G^* has a directed edge from u to v
- The transitive closure provides reachability information about a digraph





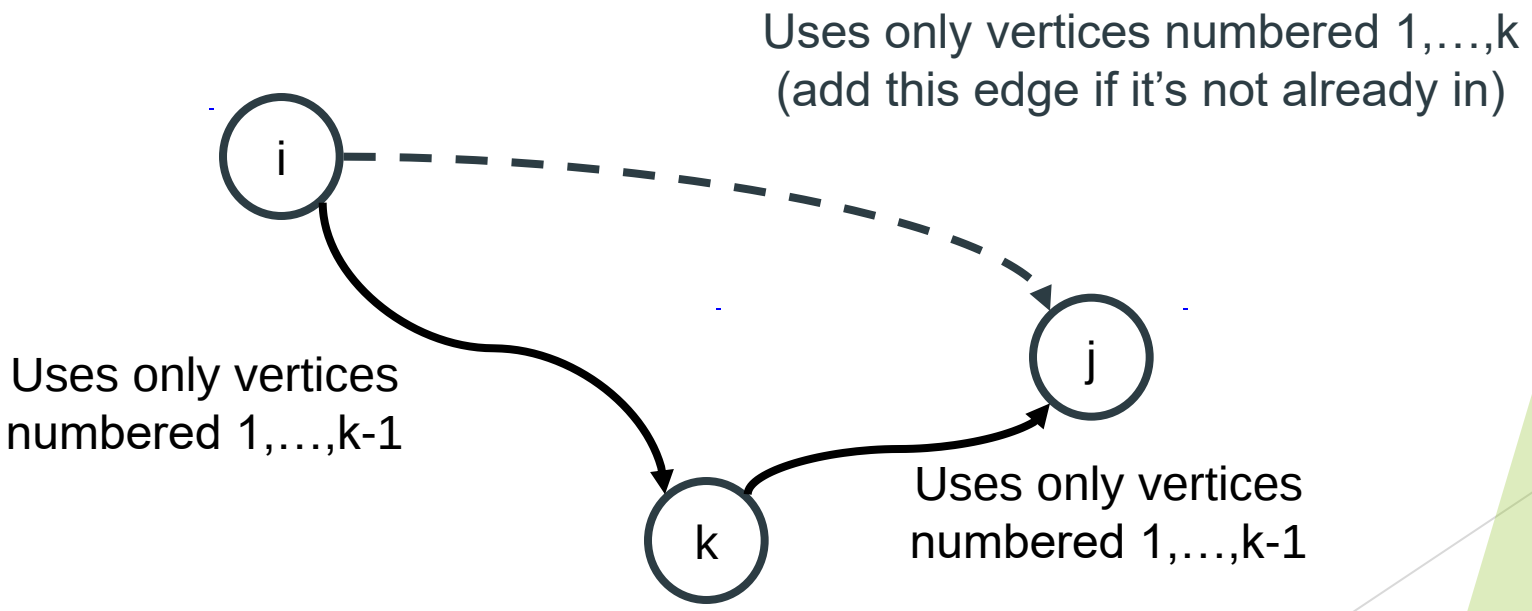
Computing the Transitive Closure

- We can perform DFS starting at each vertex
  - O(n(n+m))

- Alternatively, Use dynamic programming:
  - Floyd-Warshall Algorithm

Floyd-Warshall Transitive Closure

- Idea #1: Number the vertices 1, 2, …, n.
- Idea #2: Consider paths that use only vertices numbered 1, 2, …, k, as intermediate vertices:



However, i and j can be any vertices (other than k)

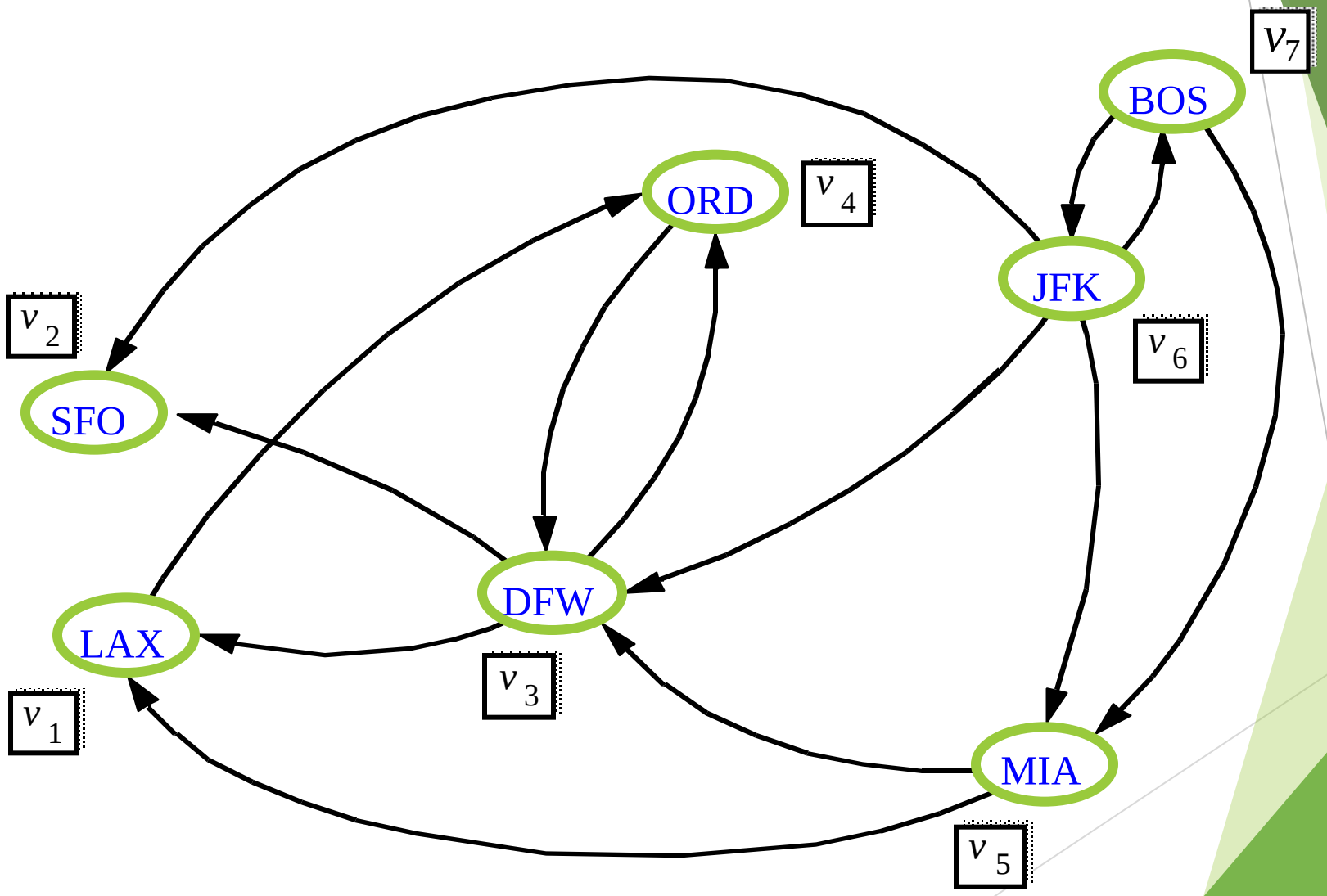
Floyd-Warshall's Algorithm

- Floyd-Warshall's algorithm numbers the vertices of G as v_1, …, v_n and computes a series of digraphs G_0, …, G_n
  - $G_0 = G$
  - G_k has a directed edge (v_i, v_j) if G_{k-1} has a directed path from v_i to v_j through v_k .
- We have that $G_n = G^*$
- In phase k, digraph G_k is computed from G_{k-1}
- Running time: $O(n^3)$, assuming areAdjacent is O(1) (e.g., adjacency matrix)
- However, the running time can be significantly improved

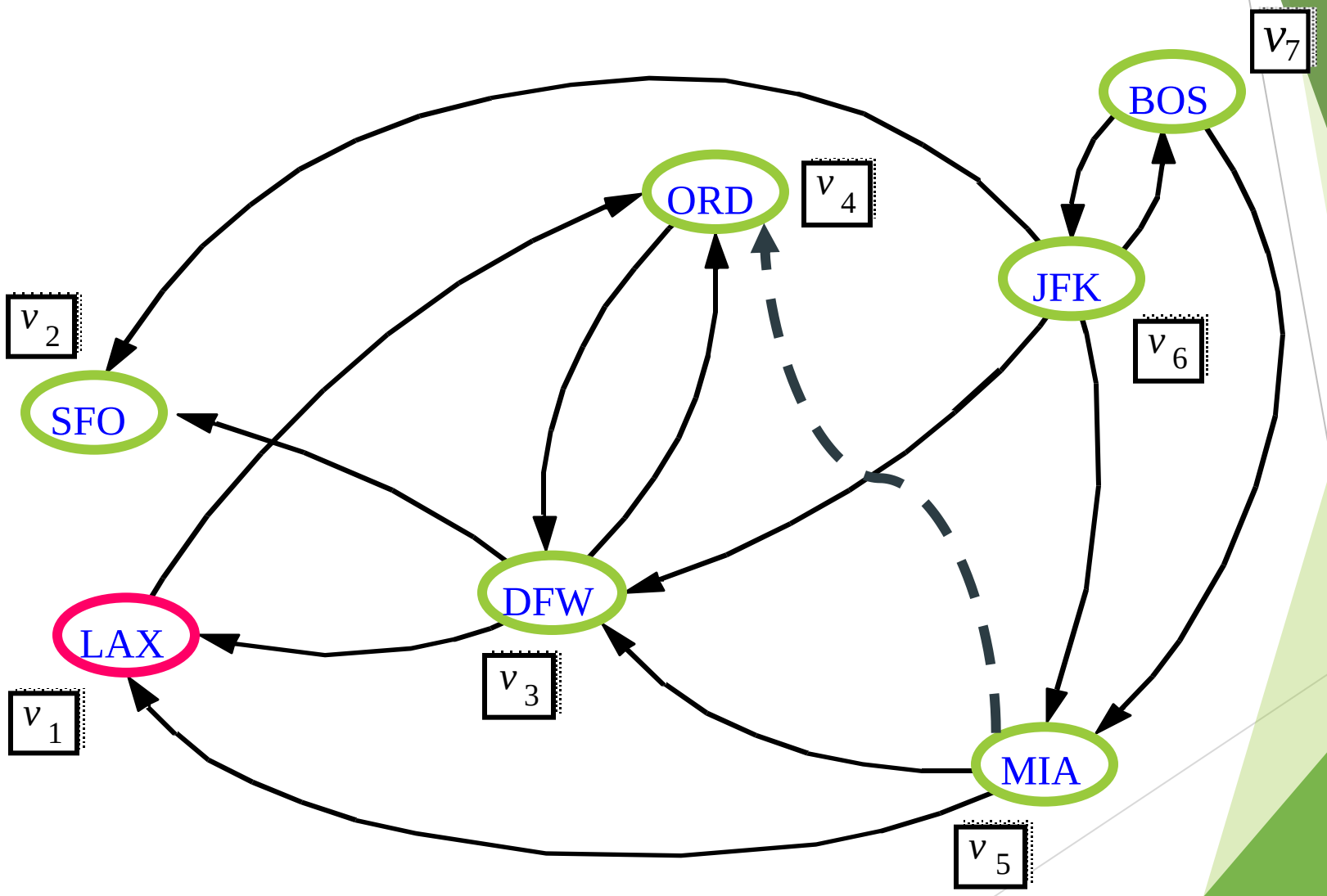
  (only consider the combination of in- and out-edges of v_k)

```
Algorithm FloydWarshall(G)
    Input digraph G
    Output transitive closure G* of G
    i ← 1
    for all v ∈ G.vertices()
        denote v as v_i
        i ← i + 1
    G_0 ← G
    for k ← 1 to n do
        G_k ← G_{k-1}
        for i ← 1 to n (i ≠ k) do
            for j ← 1 to n (j ≠ i, k) do
                if G_{k-1}.areAdjacent(v_i, v_k) ∧
                        G_{k-1}.areAdjacent(v_k, v_j)
                    if ¬G_k.areAdjacent(v_i, v_j)
                        G_k.insertDirectedEdge(v_i, v_j, k)
        return G_n
```

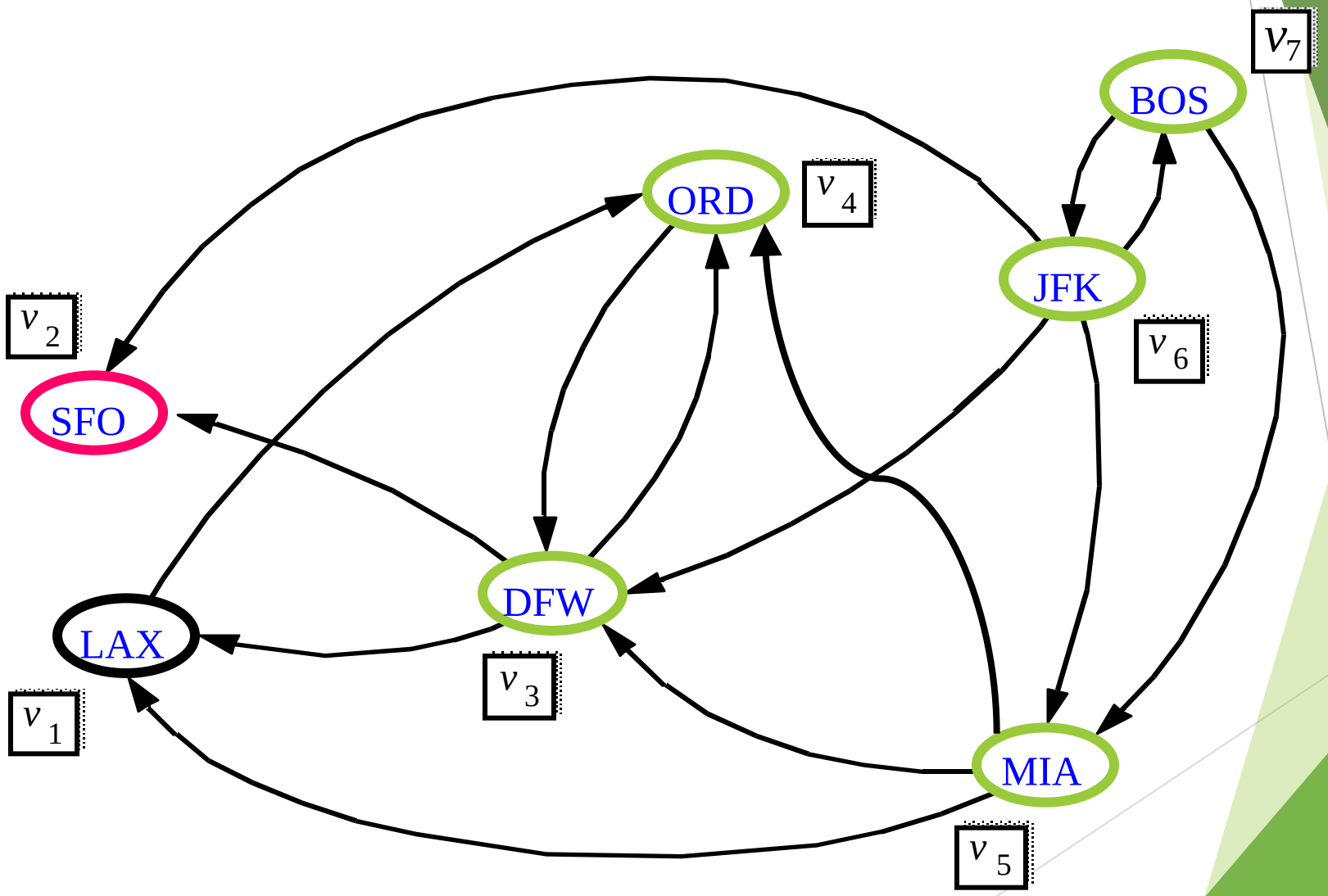
Floyd-Warshall Example



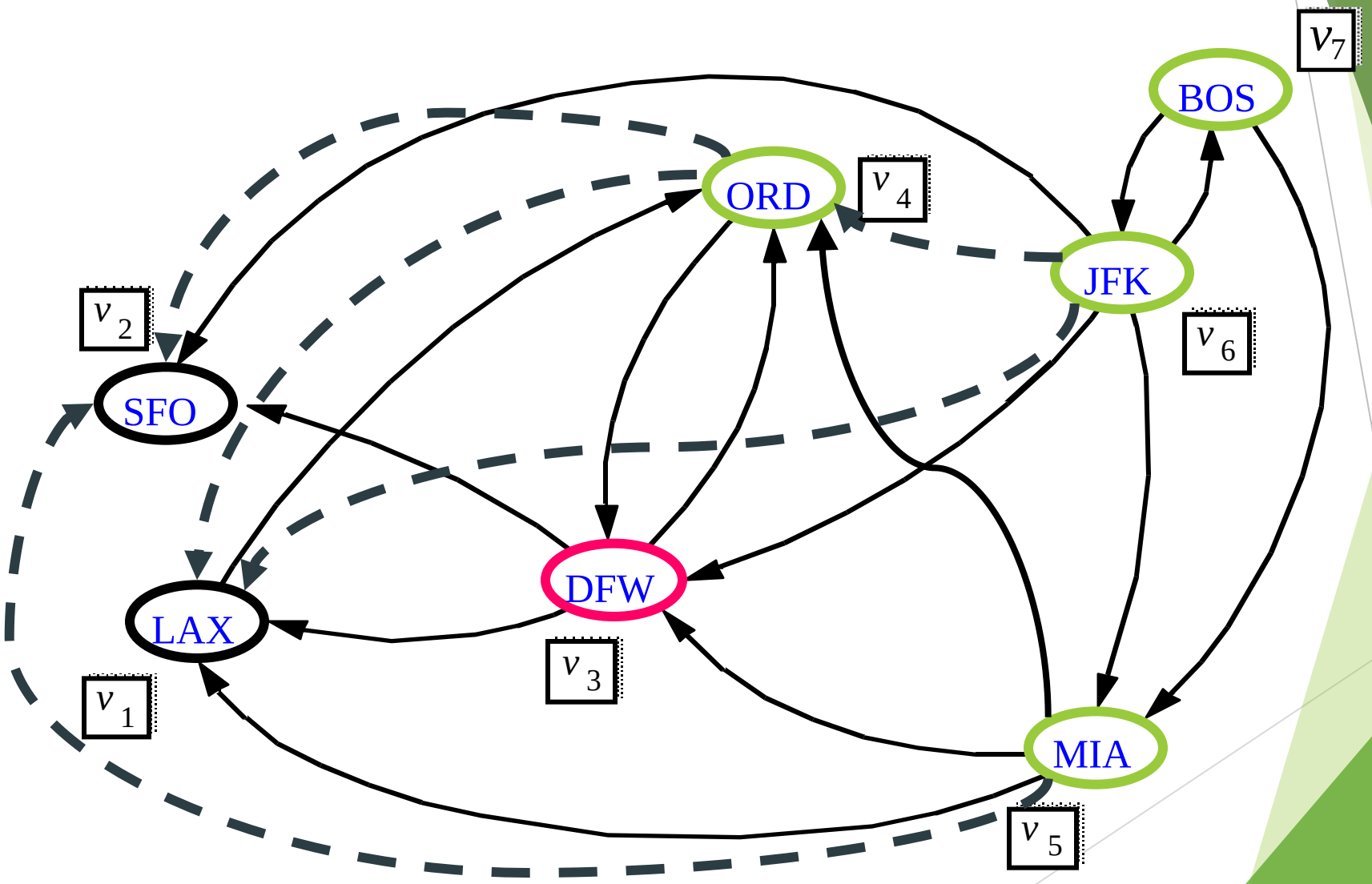
Floyd-Warshall, Iteration 1



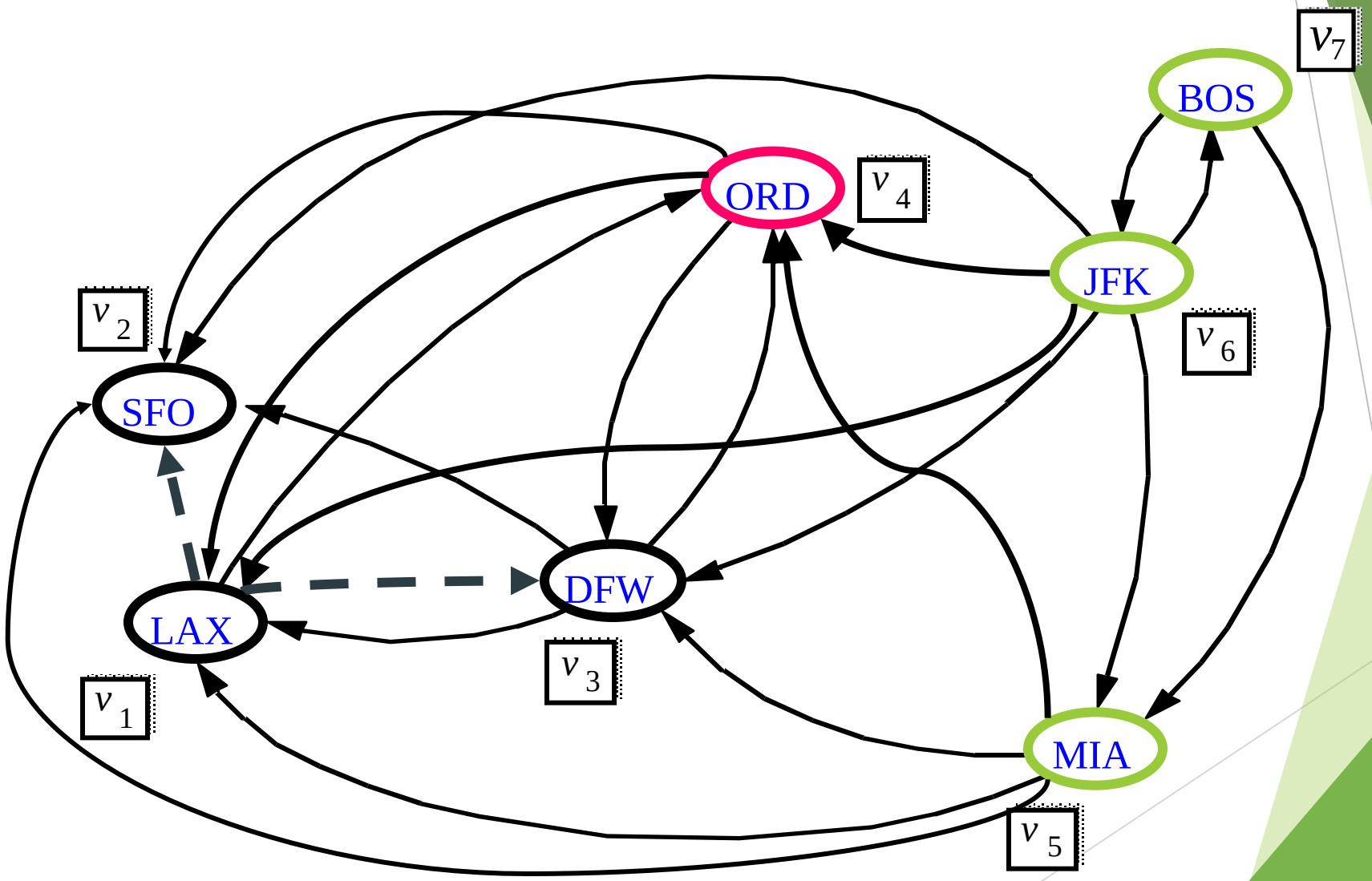
Floyd-Warshall, Iteration 2



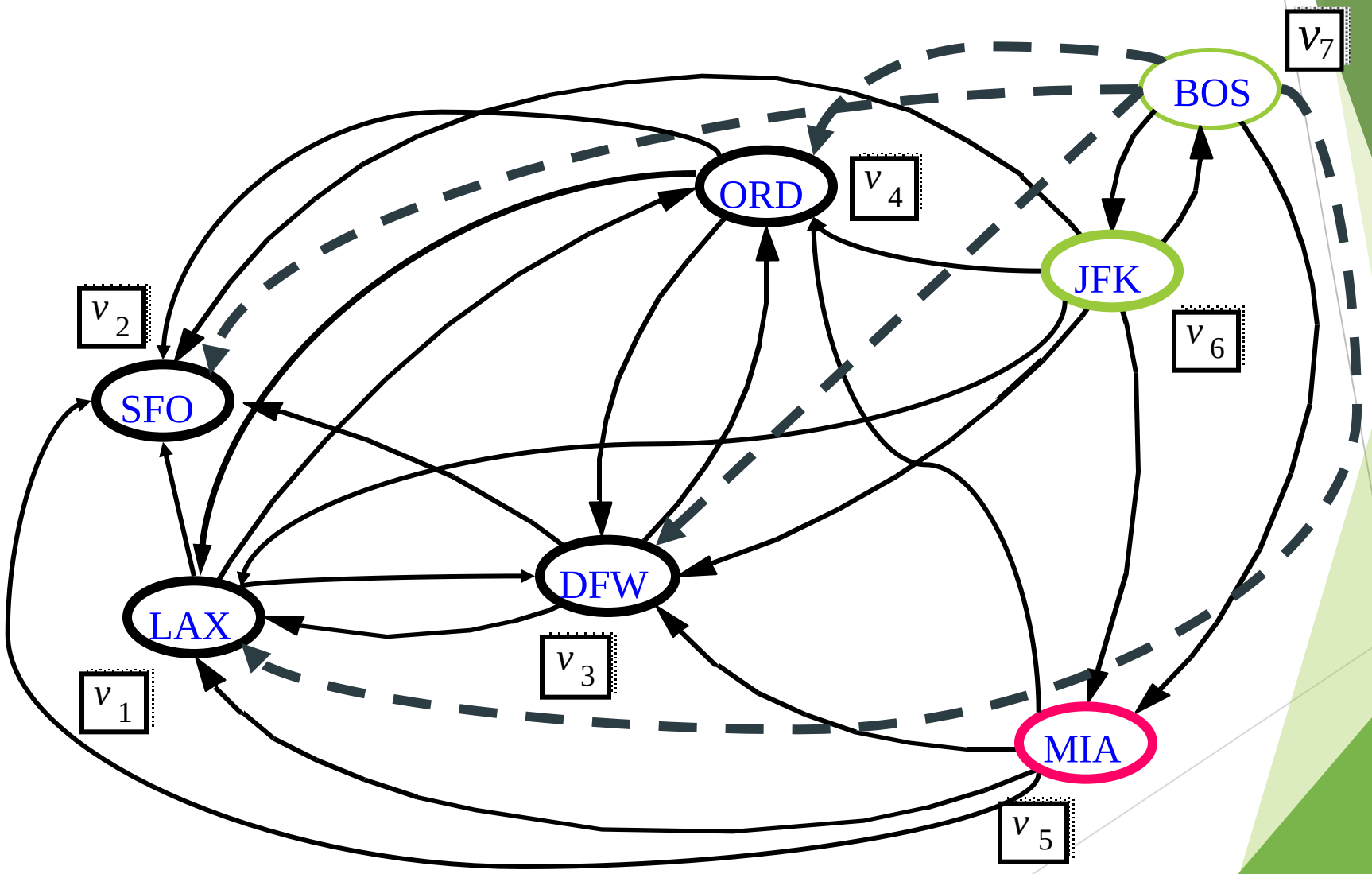
Floyd-Warshall, Iteration 3



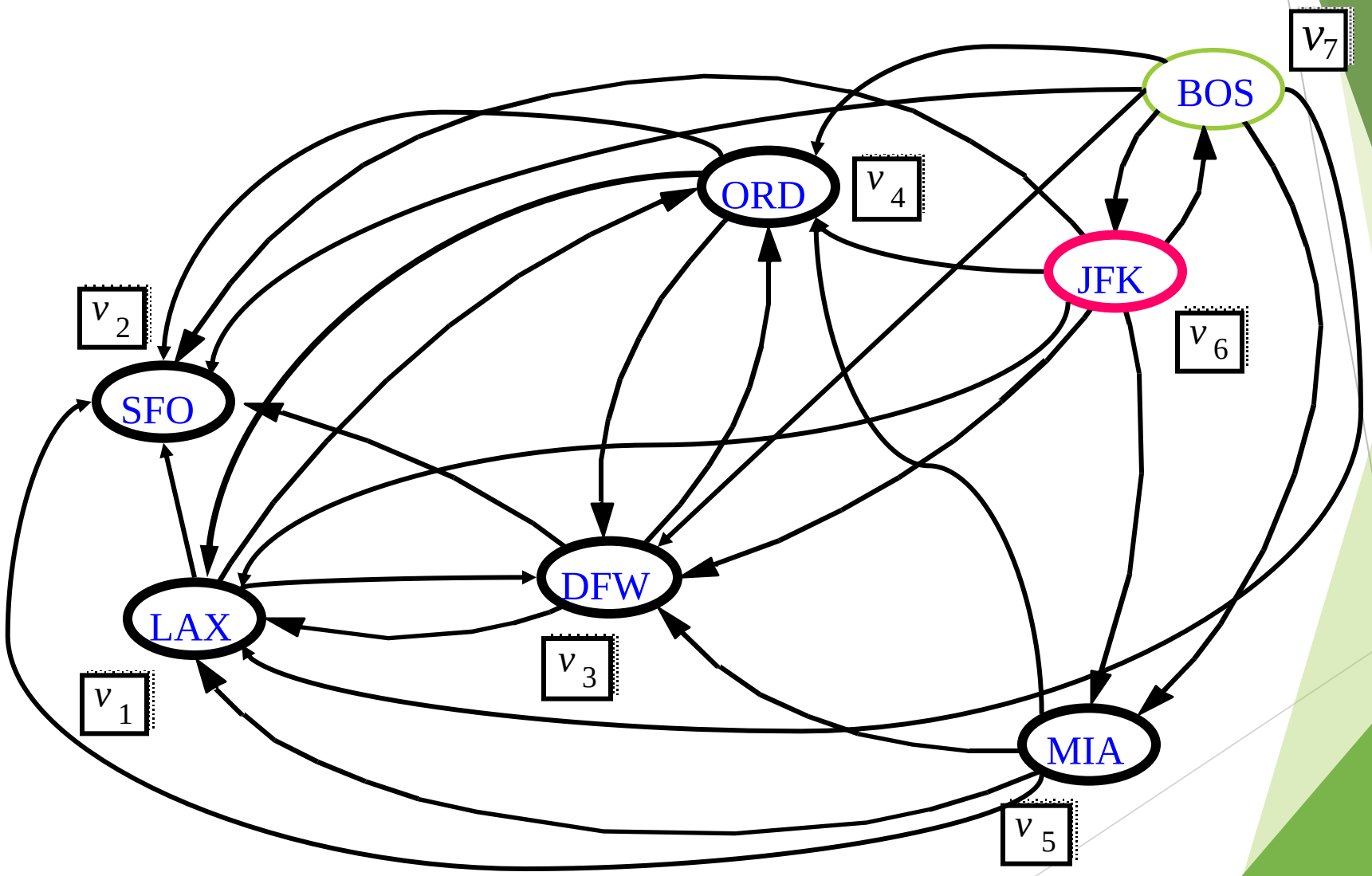
Floyd-Warshall, Iteration 4



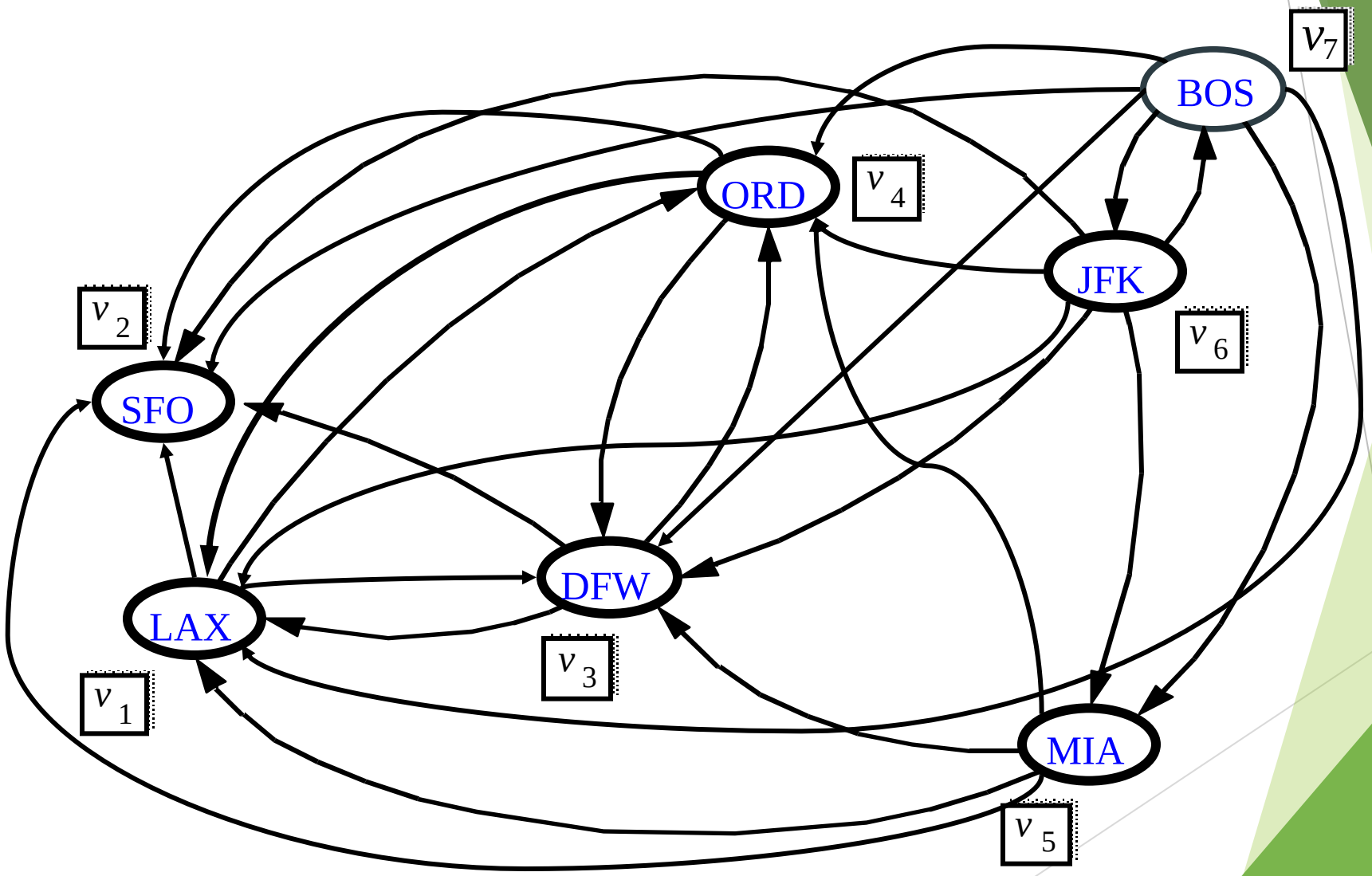
Floyd-Warshall, Iteration 5



Floyd-Warshall, Iteration 6



Floyd-Warshall, Conclusion



DAGs and Topological Ordering

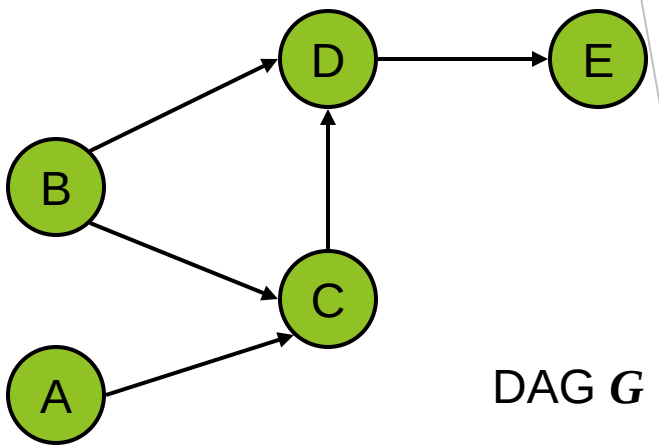
- A *directed acyclic graph* (DAG) is a digraph that has no directed cycles
- A topological ordering of a digraph is a numbering

  $$v_1, \ldots, v_n$$

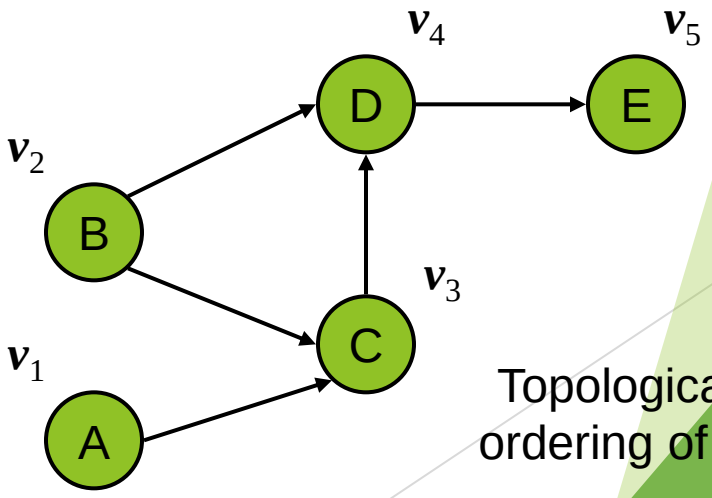
  of the vertices such that for every edge (v_i, v_j), we have $i < j$
- Example: in a task scheduling digraph, a topological ordering a task sequence that satisfies the precedence constraints

Theorem

A digraph admits a topological ordering if and only if it is a DAG



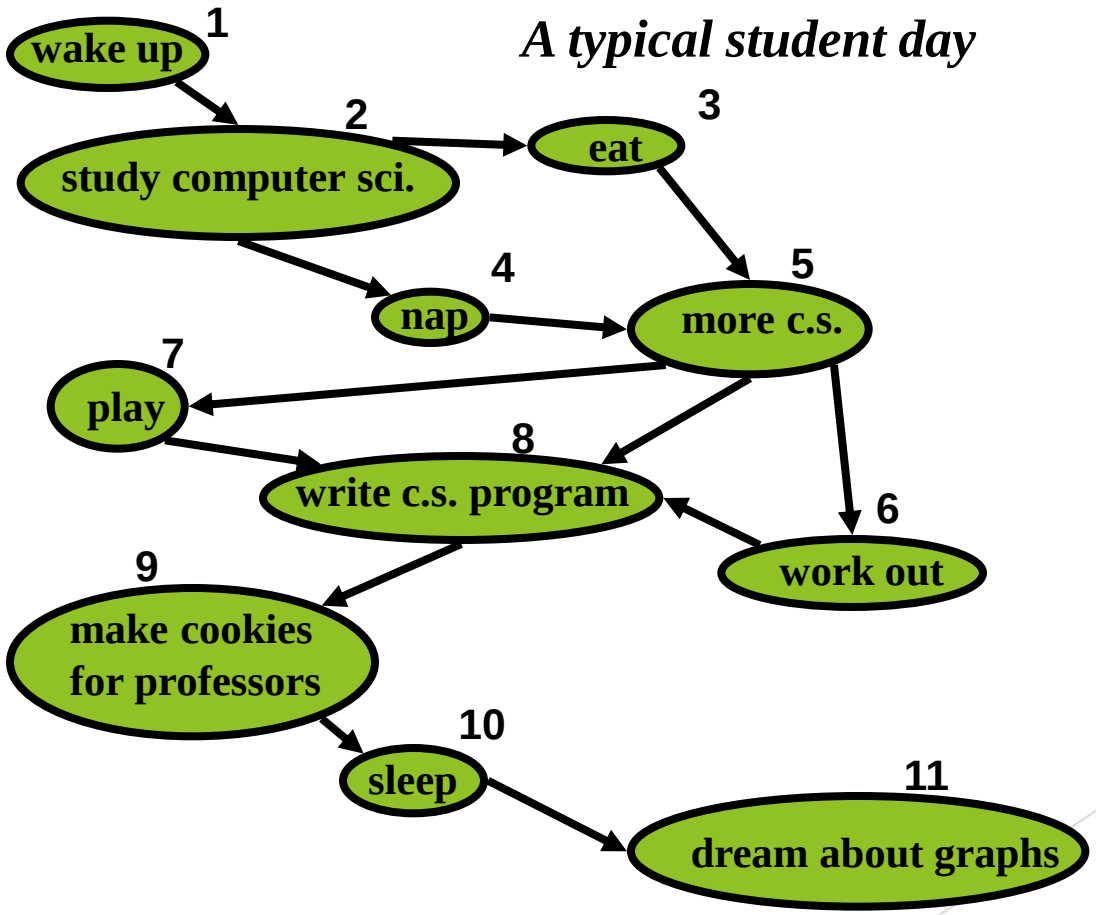
DAG G



Topological ordering of G

Topological Sorting

- Number vertices, so that (u,v) in E implies u < v



Algorithm for Topological Sorting

- Note: This algorithm is different from the one in the textbook

```
Method TopologicalSort (G)
    H ← G    // Temporary copy of G
    n ← G.numVertices()
    while H is not empty do
      Let v be a vertex with no outgoing edges
      (there must be such a vertex. Why ?)
      Label v ← n
      n ← n - 1
      Remove v and associated edges from H
```

- Running time: O(n + m).

Topological Sorting Algorithm using DFS

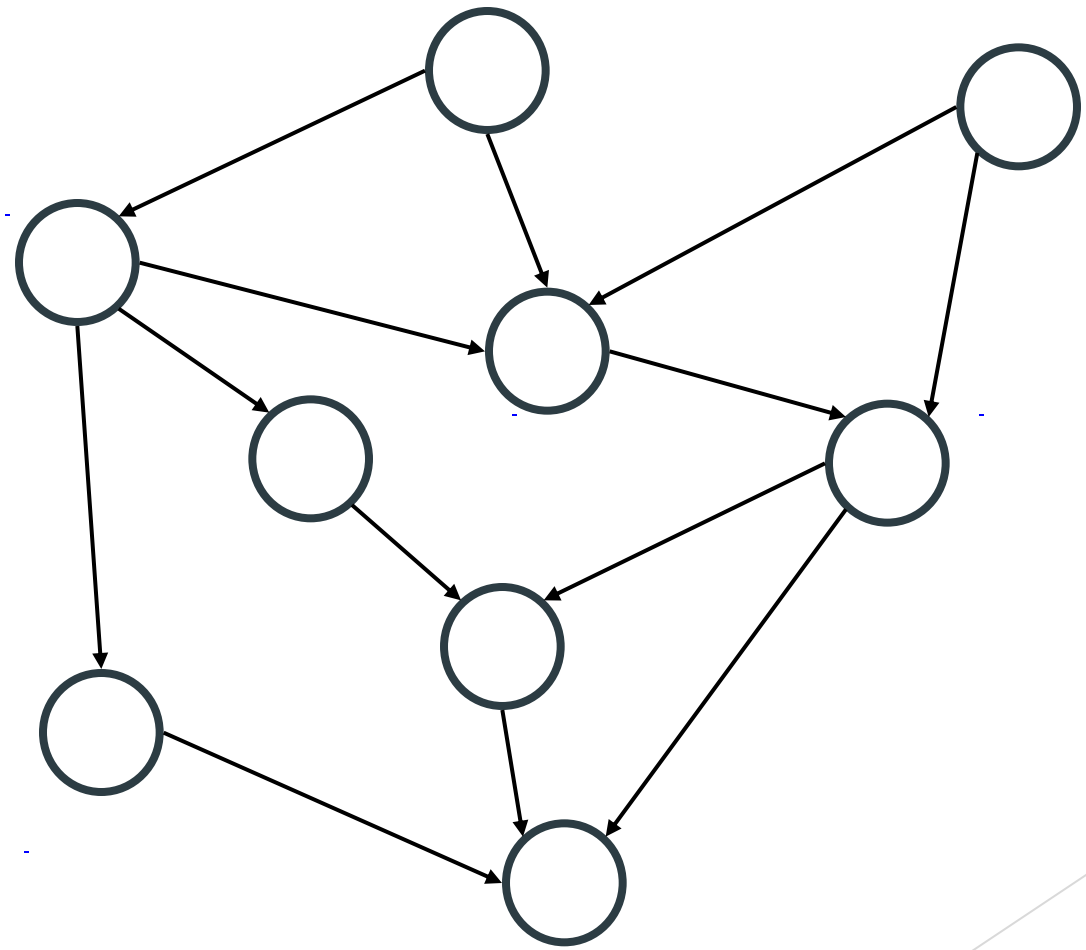
- Simulate the algorithm by using depth-first search

```
Algorithm topologicalDFS(G)
    Input dag G
    Output topological ordering of G
  n ← G.numVertices()
  for all u ∈ G.vertices()
      setLabel(u, UNEXPLORED)
  for all e ∈ G.edges()
      setLabel(e, UNEXPLORED)
  for all v ∈ G.vertices()
    if getLabel(v) = UNEXPLORED
        & outdeg(v) = 0
        topologicalDFS(G, v)
```

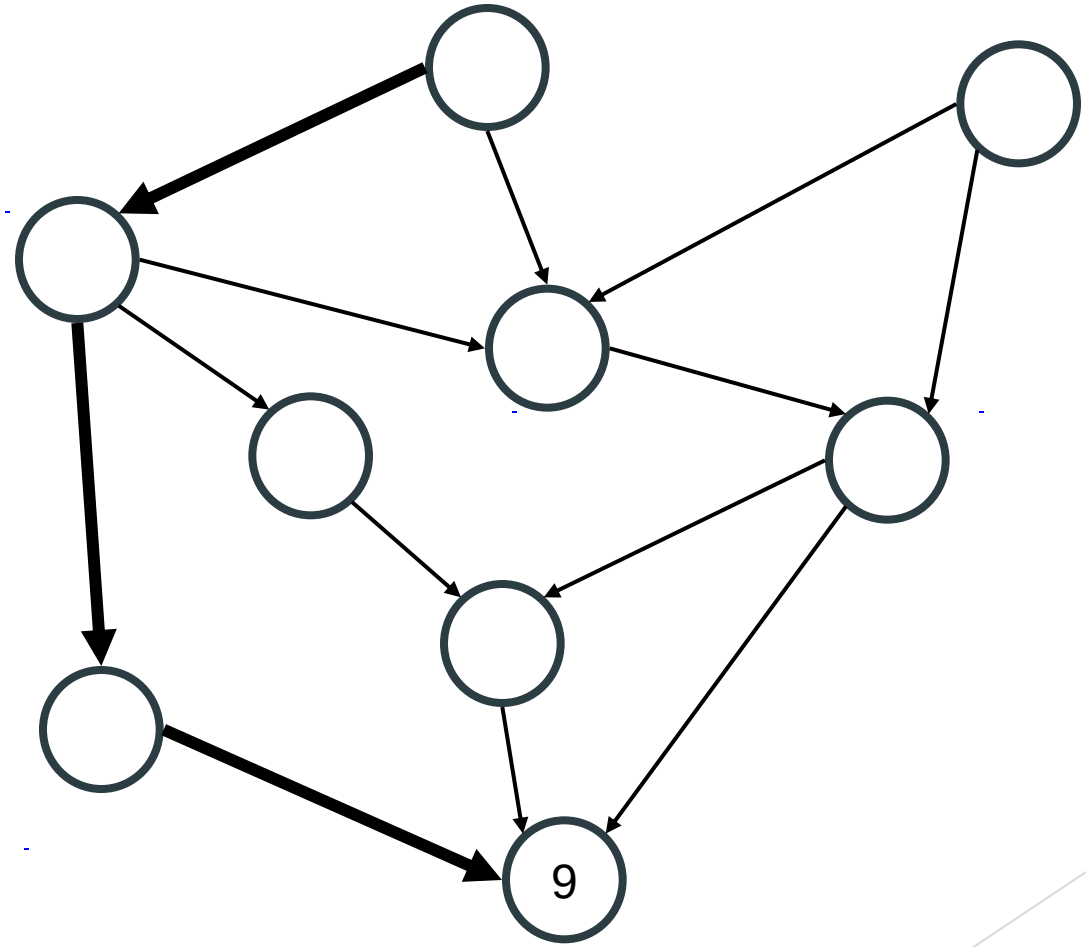
```
Algorithm topologicalDFS(G, v)
  Input graph G and a start vertex v of G
  Output labeling of the vertices of G
     in the connected component of v
  setLabel(v, VISITED)
  Label v with topological number n
   n ← n – 1 // n is a global variable
  for all e ∈ G.inIncidentEdges(v)
     if getLabel(e) = UNEXPLORED
        w ← opposite(v,e)
        if getLabel(w) = UNEXPLORED
           & outdeg(v) = 0
           setLabel(e, DISCOVERY)
           topologicalDFS(G, w)
        else
           {e is a forward or cross edge}
```

- outdeg(v): number of out-Incident UNEXPLORED edges
- O(n+m) time.

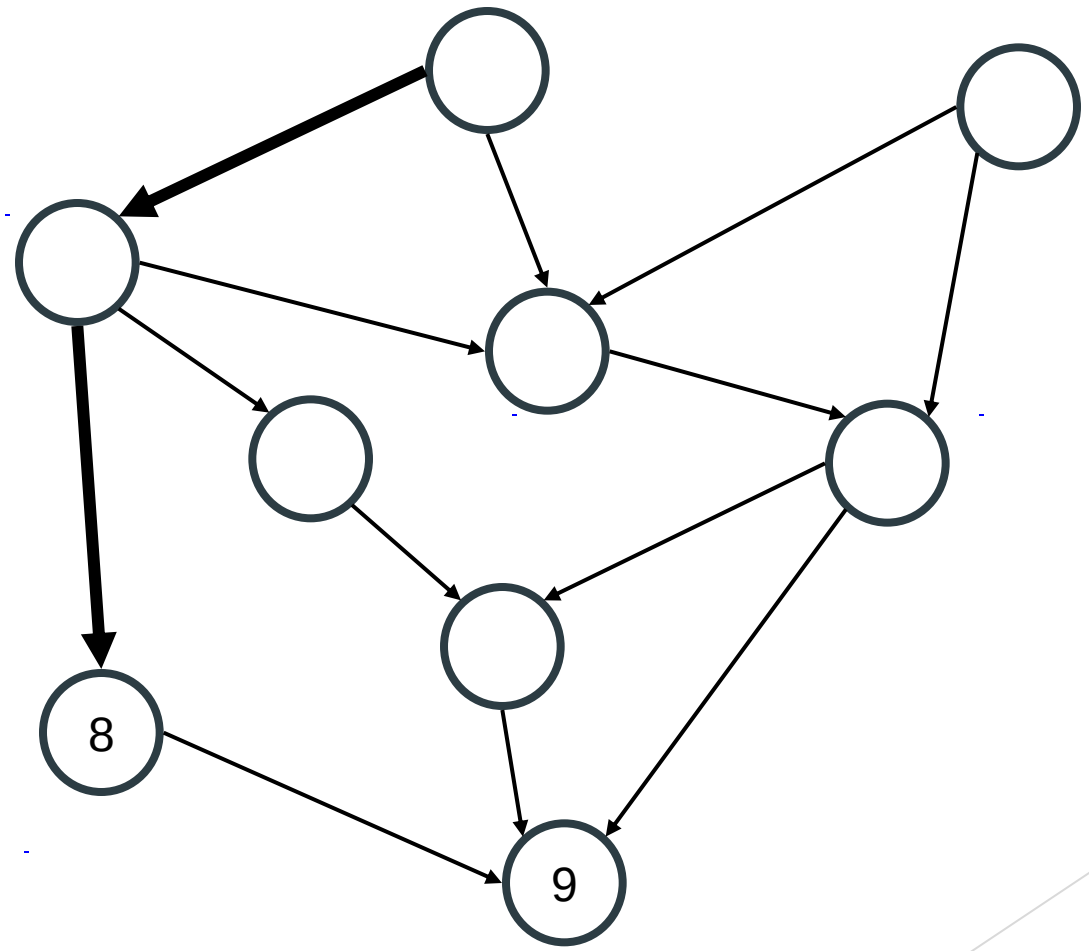
Topological Sorting Example



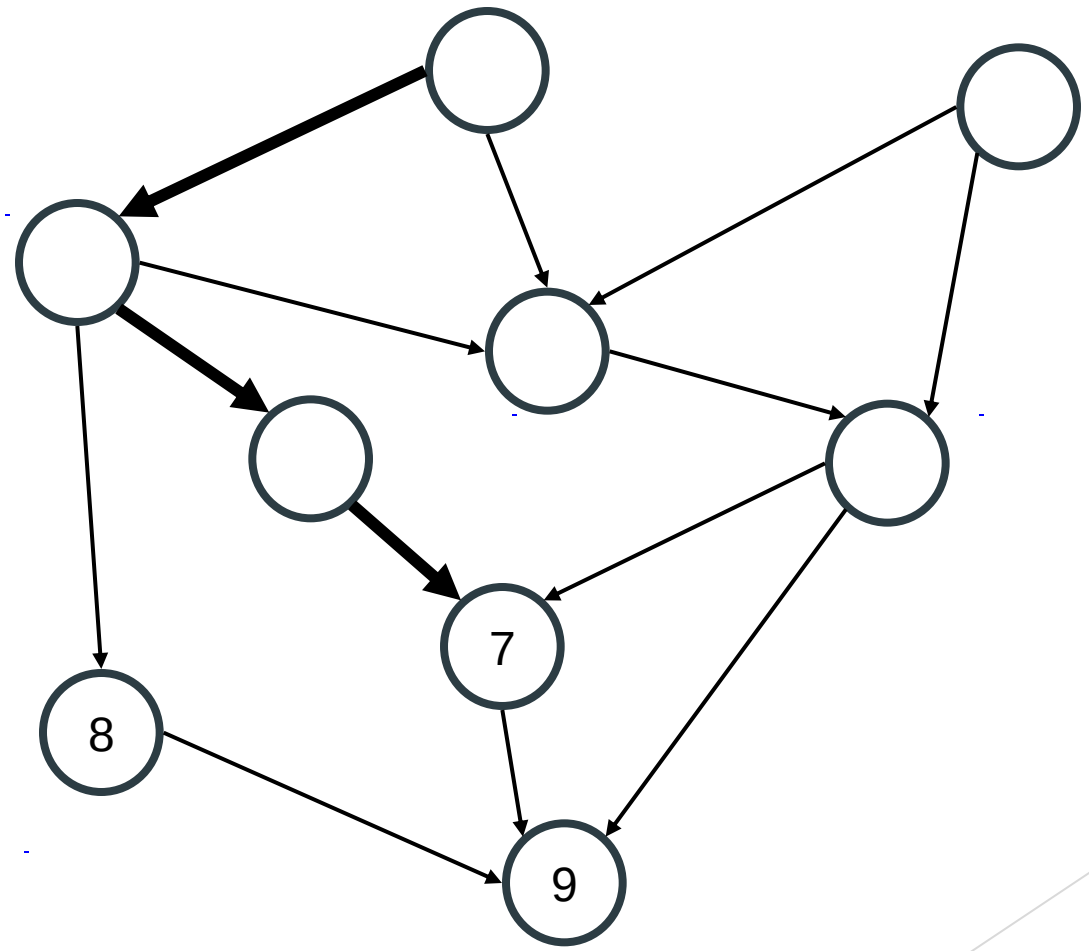
Topological Sorting Example



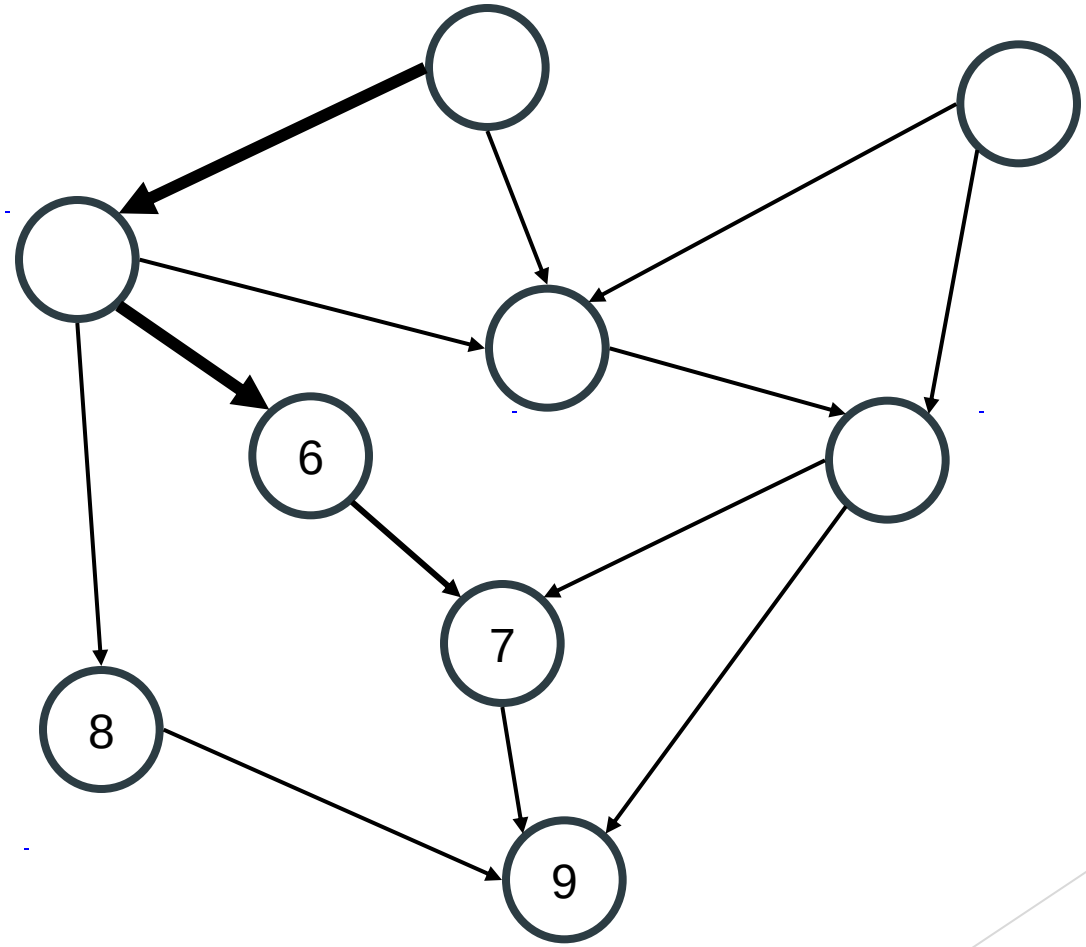
Topological Sorting Example



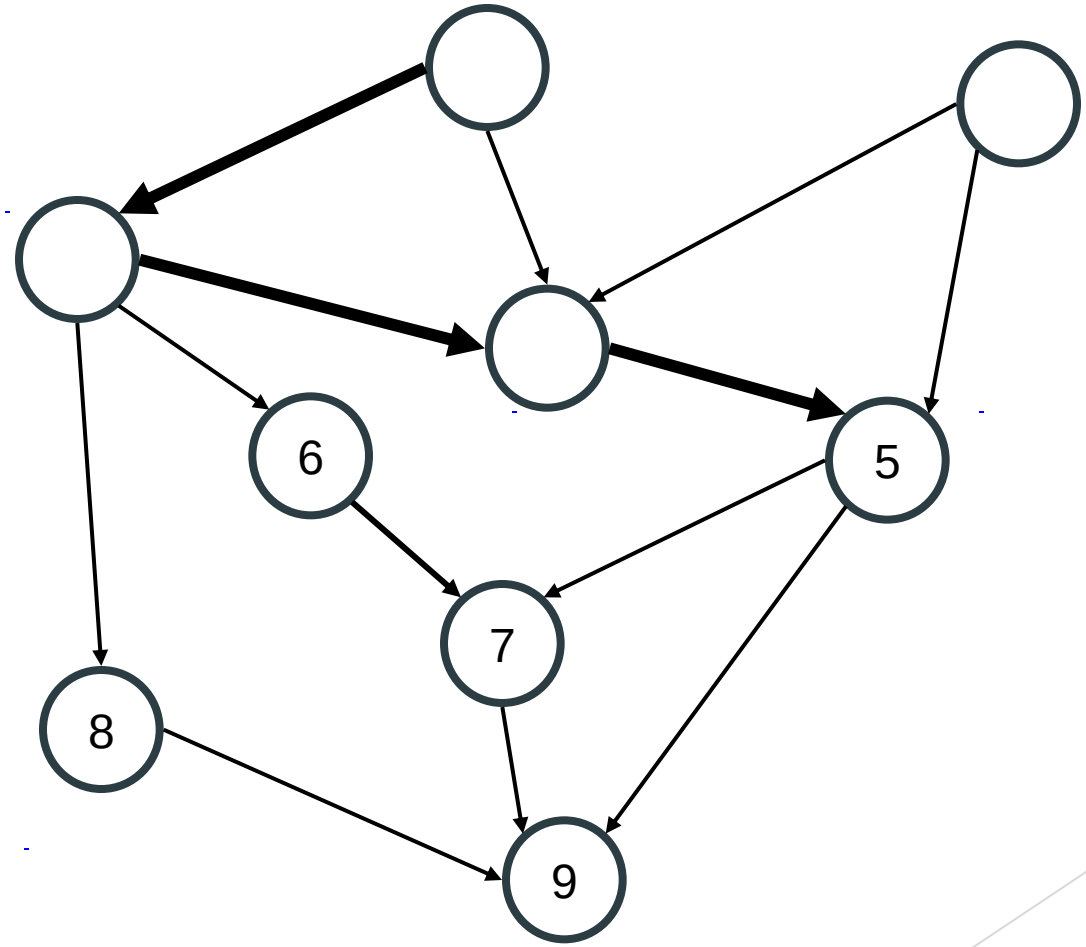
Topological Sorting Example



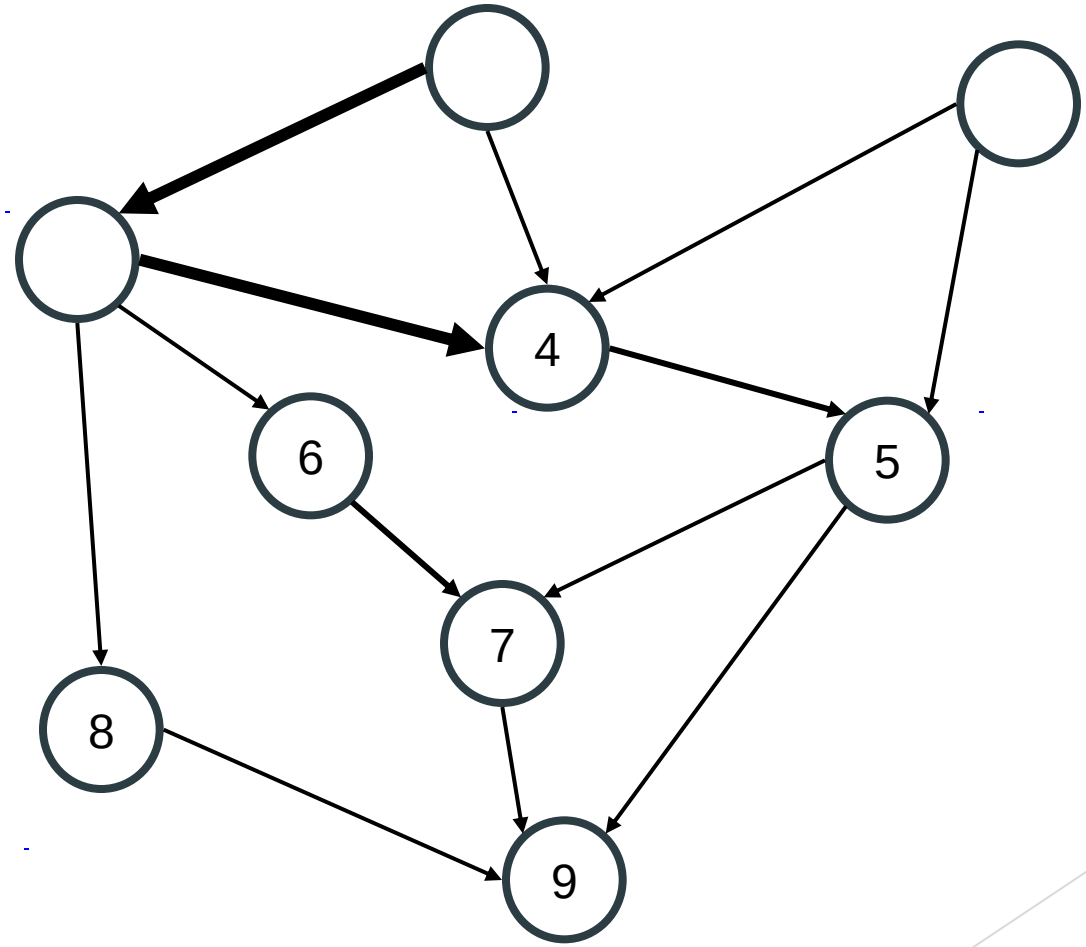
Topological Sorting Example



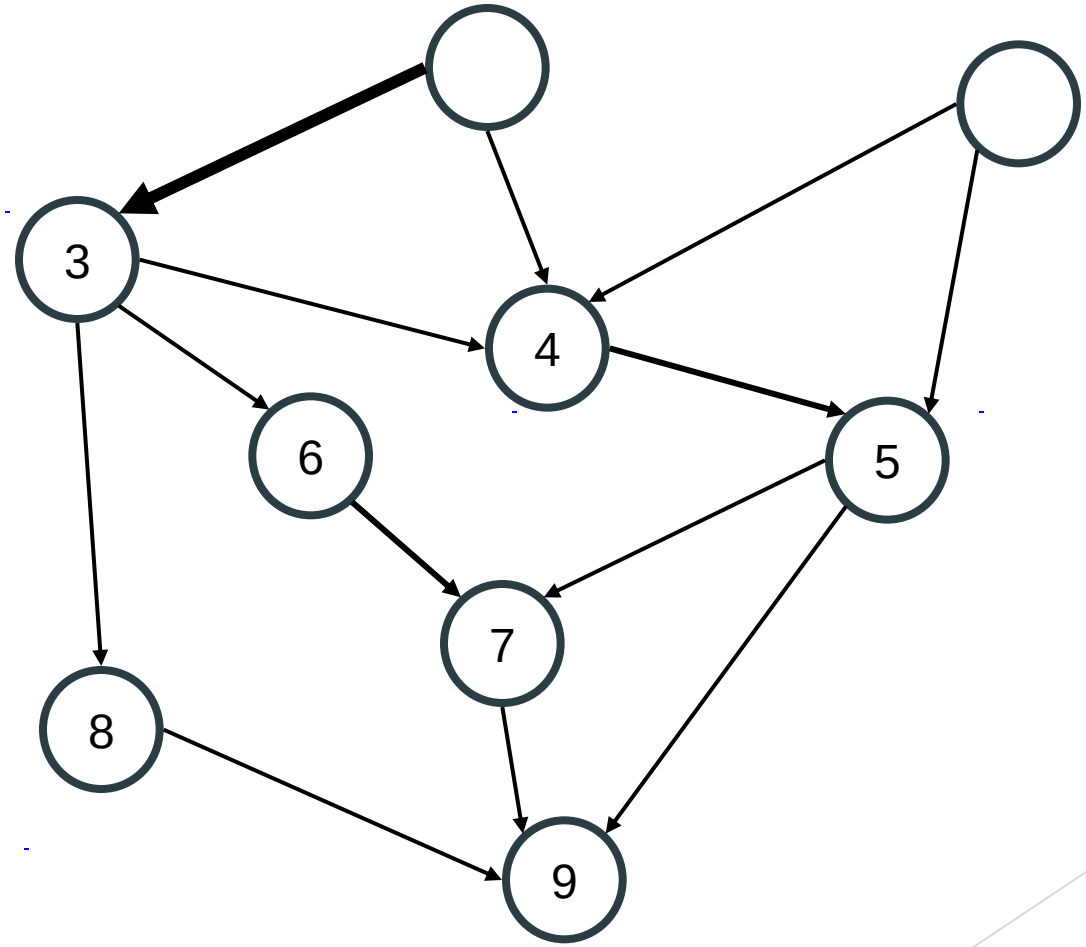
Topological Sorting Example



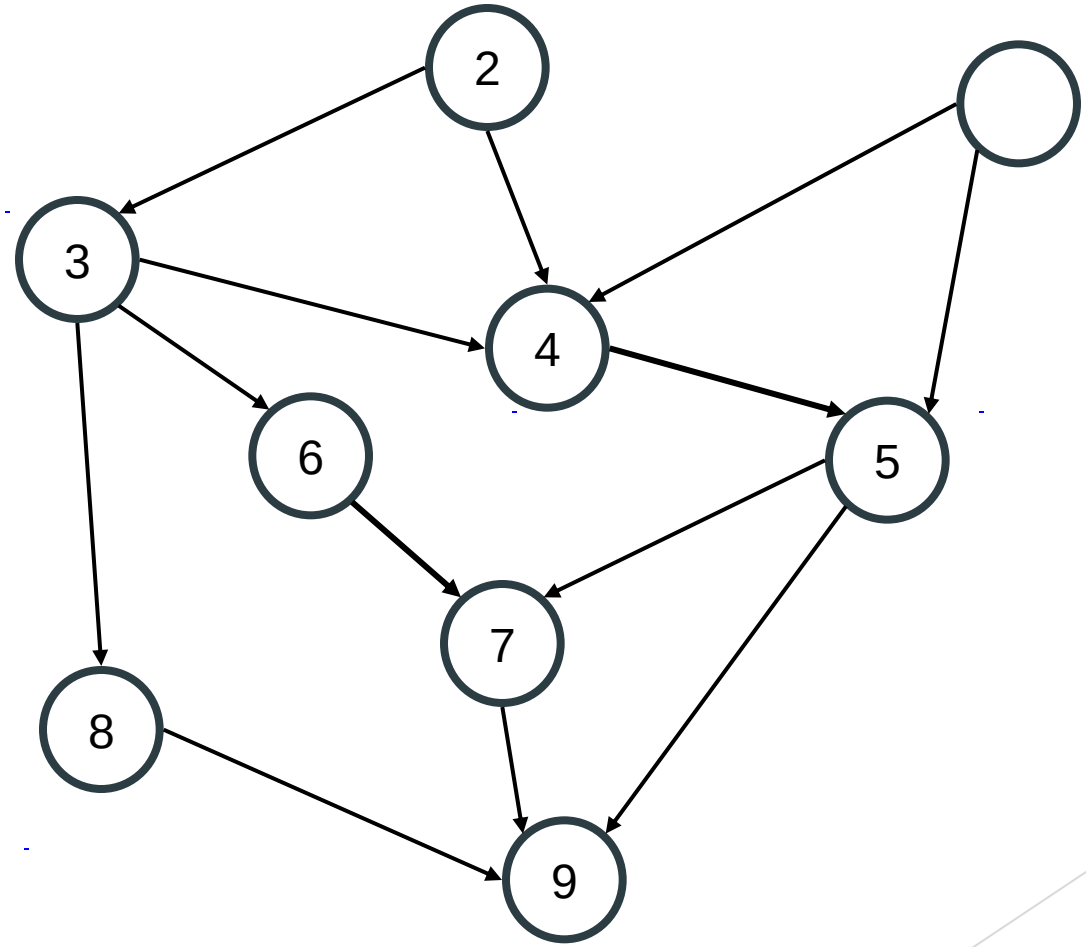
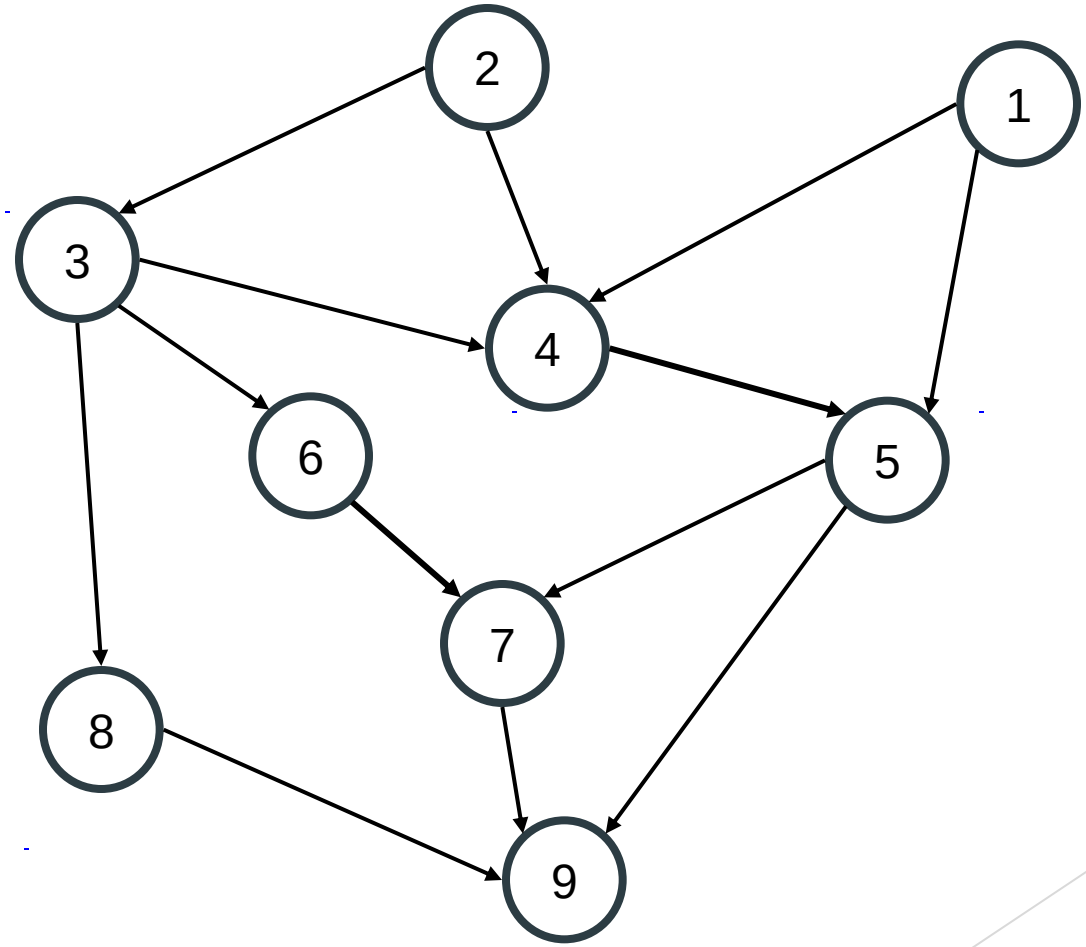
Topological Sorting Example



Topological Sorting Example



Topological Sorting Example



Topological Sorting Example